Федеральное государственное автономное образовательное
учреждение высшего образования
«Научно-образовательная корпорация ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа по «Тестированию программного обеспечения» №1

Вариант 368849

Выполнил:
Степанов Арсений Алексеевич

Группа:
P3318

Преподаватель:
Егошин Алексей Васильевич

Санкт-Петербург, 2026г.

Задание

1. Для функции $\sec(x)$ провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.

2. Провести модульное тестирование сортировки пузырьком. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.

3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели:

Артур неожиданно почувствовал, как на его бесплотной голове зашевелились волосы, когда он начал медленно, но неуклонно двигаться к пульту, но это был всего лишь, как он тут же догадался, наезд камеры при съемке.

Выполнение

Реализация и тестирование функции секанса

Функция секанса выглядит как $\sec(x) = \frac{1}{\cos(x)}$, соответственно используя ряд Маклорена можно расчитать значение функции $\cos(x)$ в окрестностях нуля и на основе этого вычислить значение секанса

$$\cos x = \sum_{n=0} (-1)^n \frac{x^{2n}}{(2n)!} \qquad \sec x = \frac{1}{\cos x}$$

Реализация функции секанса:

```
using defaults::DEFAULT_TERMS;
using precision::Eps;

template <typename T>
constexpr T abs(T x) {
    return x < T{0} ? -x : x;
}

template <typename T>
requires std::floating_point<T>
constexpr T cos(T x, size_t terms = DEFAULT_TERMS<T>) {
    T result{1.0};
    T current_x{1.0};
    int16_t sign = 1;
    T fac = 1;

    for (size_t it = 1; it < terms; ++it) {
        current_x *= x * x;
        fac *= (2 * it - 1) * (2 * it);
        sign *= -1;
        if (std::isnan(current_x) || std::isnan(fac) || std::isnan(result + sign * current
            break;
        result += sign * current_x / fac;
    }

    return result;
}

template <typename T>
requires std::integral<T>
constexpr float cos(T x, size_t terms = DEFAULT_TERMS<T>) {
    return cos(static_cast<float>(x), terms);
}

template <typename T>
requires std::floating_point<T>
constexpr T sec(T x, size_t terms = DEFAULT_TERMS<T>) {
    if (std::isinf(x) || std::isnan(x)) {
        return std::numeric_limits<T>::quiet_NaN();
    }

    if (x < 0) {
```

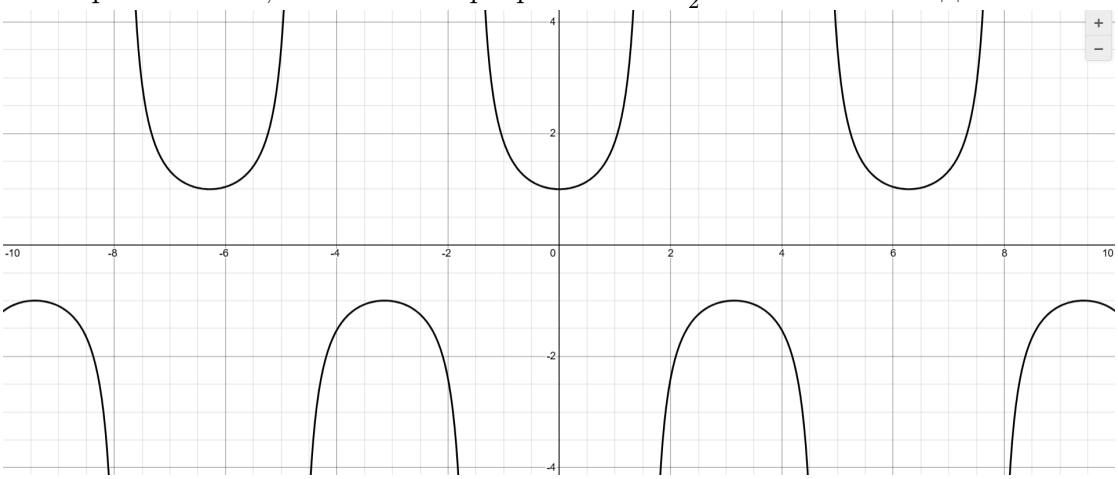
```
        x = -x;
    }

    constexpr T PI = std::numbers::pi_v<T>;
    auto pi_shift = static_cast<int32_t>(x / (PI * 2));
    x += -pi_shift * 2 * PI;

    if (abs(x - PI / 2) < Eps<T>::VALUE || abs(x - 3 * PI / 2) < Eps<T>::VALUE) {
        return std::numeric_limits<T>::quiet_NaN();
    }
    return 1 / cos(x, terms);
}

template <typename T>
requires std::integral<T>
constexpr float sec(T x, size_t terms = DEFAULT_TERMS<T>) {
    return sec(static_cast<float>(x), terms);
}
```

Для проверки работоспособности функции секанса я решил использовать разбиение на классы эквивалентности, функция секанса является периодической с интервалом в 2π, имеет точки разрыва в $x = \frac{\pi}{2} + \pi k$ и имеет вид:



На основе этого я выделил следующие классы для тестирования:

- Значения функции в точках разрыва $x = \dfrac{\pi}{2} + \pi k$
- Значения функции в точках экстремума $x = \pi k$
- Значения функции в интервале возрастания $x \in (2\pi k, \dfrac{\pi}{2} + 2\pi k)$
- Значения функции в интервале убывания $x \in (\dfrac{\pi}{2} + 2\pi k, \pi + 2\pi k)$
- Значения функции в интервале возрастания $x \in (\pi + 2\pi k, \dfrac{3\pi}{2} + 2\pi k)$
- Значения функции в интервале убывания $x \in (\dfrac{3\pi}{2} + 2\pi k, 2\pi + 2\pi k)$
- Значения функции при положительном и отрицательном аргументах

А так же дополнительные проверки краевых случаев и специфики работы с типами с плавающей запятой:

- Проверка что при подаче на вход значений NaN, функция так же вернёт NaN
- Проверка что при использовании типа float сохраняется точность как минимум до 3-ёх знаков после запятой
- Проверка что при использовании типа double сохраняется точность как минимум до 5-ти знаков после запятой
- Проверка что при использовании типа long double сохраняется точность как минимум до 7-ми знаков после запятой

Таблица используемых и ожидаемых значений по классам эквивалентности

Класс эквивалентности	**Используемые значения**	**Ожидаемое значение / поведение**
Точки разрыва $x = \frac{\pi}{2} + \pi k$	$-\frac{\pi}{2}$, $\frac{\pi}{2}$, $\frac{3\pi}{2}$	`NaN`
Окрестность точек разрыва	$-\frac{\pi}{2} \pm \varepsilon$, $\frac{\pi}{2} \pm \varepsilon$	Значение определено (не `NaN`)
Точки экстремума $x = \pi k$	$-\pi$ 0 π	-1 1 -1
Интервал возрастания $x \in (0, \frac{\pi}{2})$	0.228 0.322 1.337	1.02656714459, 1.05418023973, 4.31644314933
Интервал убывания $x \in (\frac{\pi}{2}, \pi)$	1.7 1.812 2.28	-7.76129399605, -4.18634900555, -1.53555659487
Интервал возрастания $x \in (\pi, \frac{3\pi}{2})$	3.22 4.2 4.6	-1.00308174955, -2.03973060149, -8.91642861136
Интервал убывания $x \in (\frac{3\pi}{2}, 2\pi)$	4.88 5.0 5.13	5.99422183928, 3.52532008582, 2.46561733623
Периодичность функции $T = 2\pi$	$x_0 = 0.1459$, $x_0 \pm 20\pi$	$\sec(x) = \sec(x_0)$
Специальные значения с плавающей точкой	$+\infty$, $-\infty$, `NaN`	`NaN`

Реализация тестов:

```
#include <gtest/gtest.h>
#include <cmath>
#include <numbers>
#include <type_traits>

#include <maths/maths.hpp>
#include <maths/defaults.hpp>

using arms::maths::sec;
using arms::maths::precision::Eps;

```

```
template <typename T>
class SecTest : public testing::Test {};

using FloatingPointTypes = testing::Types<float, double, long double>;
TYPED_TEST_SUITE(SecTest, FloatingPointTypes);

TYPED_TEST(SecTest, TestPointsOfDiscontinuity) {
    using T = TypeParam;
    constexpr auto PI = std::numbers::pi_v<T>;

    EXPECT_TRUE(std::isnan(sec(-PI / 2)));
    EXPECT_TRUE(std::isnan(sec(PI / 2)));
    EXPECT_TRUE(std::isnan(sec(3 * PI / 2)));
}

TYPED_TEST(SecTest, TestDiscontinuityNeighborhoodRegion) {
    using T = TypeParam;
    constexpr auto PI = std::numbers::pi_v<T>;

    EXPECT_FALSE(std::isnan(sec(-PI / 2 - Eps<T>::VALUE)));
    EXPECT_FALSE(std::isnan(sec(-PI / 2 + Eps<T>::VALUE)));
    EXPECT_FALSE(std::isnan(sec(PI / 2 - Eps<T>::VALUE)));
    EXPECT_FALSE(std::isnan(sec(PI / 2 + Eps<T>::VALUE)));
}

TYPED_TEST(SecTest, TestInfinity) {
    using T = TypeParam;
    constexpr auto INF = std::numeric_limits<T>::infinity();

    EXPECT_TRUE(std::isnan(sec(INF)));
    EXPECT_TRUE(std::isnan(sec(-INF)));
}

TYPED_TEST(SecTest, TestZero) {
    using T = TypeParam;

    EXPECT_NEAR(sec(static_cast<T>(0.0)), static_cast<T>(1.0), Eps<T>::VALUE);
}

TYPED_TEST(SecTest, TestRange1) {
    using T = TypeParam;

    EXPECT_NEAR(sec(static_cast<T>(0.228)), static_cast<T>(1.02656714459), Eps<T>::VALUE);
    EXPECT_NEAR(sec(static_cast<T>(0.322)), static_cast<T>(1.05418023973), Eps<T>::VALUE);
    EXPECT_NEAR(sec(static_cast<T>(1.337)), static_cast<T>(4.31644314933), Eps<T>::VALUE);
}

TYPED_TEST(SecTest, TestRange2) {
    using T = TypeParam;

    EXPECT_NEAR(sec(static_cast<T>(1.7)), static_cast<T>(-7.76129399605), Eps<T>::VALUE);
    EXPECT_NEAR(sec(static_cast<T>(1.812)), static_cast<T>(-4.18634900555), Eps<T>::VALUE)
    EXPECT_NEAR(sec(static_cast<T>(2.28)), static_cast<T>(-1.53555659487), Eps<T>::VALUE);
}

```

```
TYPED_TEST(SecTest, TestRange3) {
    using T = TypeParam;

    EXPECT_NEAR(sec(static_cast<T>(3.22)), static_cast<T>(-1.00308174955), Eps<T>::VALUE);
    EXPECT_NEAR(sec(static_cast<T>(4.2)), static_cast<T>(-2.03973060149), Eps<T>::VALUE);
    EXPECT_NEAR(sec(static_cast<T>(4.6)), static_cast<T>(-8.91642861136), Eps<T>::VALUE);
}

TYPED_TEST(SecTest, TestRange4) {
    using T = TypeParam;

    EXPECT_NEAR(sec(static_cast<T>(4.88)), static_cast<T>(5.99422183928), Eps<T>::VALUE);
    EXPECT_NEAR(sec(static_cast<T>(5)), static_cast<T>(3.52532008582), Eps<T>::VALUE);
    EXPECT_NEAR(sec(static_cast<T>(5.13)), static_cast<T>(2.46561733623), Eps<T>::VALUE);
}

TYPED_TEST(SecTest, TestExtremum) {
    using T = TypeParam;
    constexpr auto PI = std::numbers::pi_v<T>;

    EXPECT_NEAR(sec(-PI), -1.0, Eps<T>::VALUE);
    EXPECT_NEAR(sec<T>(0.0), 1.0, Eps<T>::VALUE);
    EXPECT_NEAR(sec(PI), -1.0, Eps<T>::VALUE);
}

TYPED_TEST(SecTest, TestIntervals) {
    using T = TypeParam;
    constexpr T INITIAL = 0.1459;
    constexpr auto PI = std::numbers::pi_v<T>;
    constexpr auto INTERVAL = 2 * PI;

    EXPECT_NEAR(sec(INITIAL), sec(INITIAL - INTERVAL * 30), Eps<T>::VALUE);
    EXPECT_NEAR(sec(INITIAL), sec(INITIAL - INTERVAL * 20), Eps<T>::VALUE);
    EXPECT_NEAR(sec(INITIAL), sec(INITIAL - INTERVAL * 10), Eps<T>::VALUE);
    EXPECT_NEAR(sec(INITIAL), sec(INITIAL + INTERVAL * 10), Eps<T>::VALUE);
    EXPECT_NEAR(sec(INITIAL), sec(INITIAL + INTERVAL * 20), Eps<T>::VALUE);
    EXPECT_NEAR(sec(INITIAL), sec(INITIAL + INTERVAL * 30), Eps<T>::VALUE);
}
```

Реализация и тестирование функции сортировки пузырьком

Реализация сортировки пузырьком:

```
template <std::random_access_iterator RandomIt, class Compare>
requires std::indirect_strict_weak_order<Compare, RandomIt>

constexpr void bubble_sort(RandomIt first, RandomIt last, Compare comp) {
    if (first == last) {
        return;
    }

    for (auto unsorted_end = last; unsorted_end != first; --unsorted_end) {
        bool swapped = false;
```

```
        for (auto it = first; (it + 1) < unsorted_end; ++it) {
            auto next = it + 1;
            if (std::invoke(comp, *next, *it)) {
                std::iter_swap(it, next);
                swapped = true;
            }
        }

        if (!swapped) {
            return;
        }
    }
}

template <std::random_access_iterator RandomIt>
constexpr void bubble_sort(RandomIt first, RandomIt last) {
    bubble_sort(first, last, std::less<>{});
}
```

Для проверки работоспособности алгоритма сортировки были выделены некоторые краевые случаи и проверка поведения алгоритма:

- Проверка того что алгоритм правильно сортриует переданный контейнер
- Проверка того что алгоритм сортирует по возрастанию по-умолчанию
- Проверка того что алгоритм корректно работает с компараторами
- Проверка того что алгоритм корректно работает с отрицательными числами и дублирующимися значениями
- Проверка того что алгоритм корректно работает на пустом контейнере
- Проверка того что алгоритм корректно работает на контейнере с одним элементом

Реализация тестирования:

```
TEST(BubbleSortTest, SortsAscendingByDefault) {
    std::vector<int> values{5, 1, 4, 2, 8};
    arms::sorting::bubble_sort(values.begin(), values.end());

    EXPECT_EQ(values, (std::vector<int>{1, 2, 4, 5, 8}));
}

TEST(BubbleSortTest, SortsDescendingWithComparator) {
    std::vector<int> values{5, 1, 4, 2, 8};
    arms::sorting::bubble_sort(values.begin(), values.end(), std::greater<>{});

    EXPECT_EQ(values, (std::vector<int>{8, 5, 4, 2, 1}));
}

TEST(BubbleSortTest, HandlesDuplicatesAndNegatives) {
    std::vector<int> values{3, -1, 3, 2, -1, 0};
    arms::sorting::bubble_sort(values.begin(), values.end());
```

```
    EXPECT_EQ(values, (std::vector<int>{-1, -1, 0, 2, 3, 3}));
}

TEST(BubbleSortTest, HandlesEmptyRange) {
    std::vector<int> values;
    arms::sorting::bubble_sort(values.begin(), values.end());

    EXPECT_TRUE(values.empty());
}

TEST(BubbleSortTest, HandlesSingleElement) {
    std::vector<int> values{42};
    arms::sorting::bubble_sort(values.begin(), values.end());

    EXPECT_EQ(values, (std::vector<int>{42}));
}
```

Разработка и тестирование доменной модели

На основе текста были выделены отдельные сущности представляющие доменную модель и предметную область. Тесты используются для проверки логической согласованности последовательности действий и внутреннего состояния предметной области. Основные сущности:

- Артур
- Голова
- Волосы
- Предположение
- Камера
- Движение к пульту
- Восприятие

Тестируются следующие явления в доменной области:

- Соответствие ожидаемым результатам при соответствие потоку событий в исходном тексте
- Проверка корректности причино-следственной связи
- Проверка правильной интерпретации числовых значений для категориальных признаков

Схема конечного автомата доменной модели

- S_0: исходное состояние (волосы неподвижны, восприятие отсутствует, причина не определена)
- S_1: параметры движения заданы (`beginMovingToConsole`)
- S_2: зафиксировано иллюзорное шевеление волос, причина — эффект наезда камеры

- S_3: событие не произошло, причина осталась не определена

Условия переходов:

- G_1: `shot.isDollyIn() == true`
- G_2: `movement.isSlow() == true`
- G_3: `movement.isSteadyApproach() == true`

```
S0 -- beginMovingToConsole --> S1
S1 -- processFilmingScene [G1 && G2 && G3] --> S2
S1 -- processFilmingScene [!(G1 && G2 && G3)] --> S3

S2: Hair=MOVING, Perception=HAIR_STIRRING(illusory), Cause=CAMERA_EFFECT
S3: Hair=STILL,  Perception=NONE,                    Cause=UNDETERMINED
```

Какие этапы и переходы покрыты тестами

Этап / переход автомата	Тесты, которые это проверяют
$S_0 \to S_1$ (задание движения к пульту)	`BaseScenarioResolvesToCameraEffect`, `NoCameraDollyNoIllusoryPerception`, `InferenceCauseIsCameraEffectWhenEventOccurs`, `CognitiveStatesFollowExpectedOrder`, `ImmaterialHeadScenarioRemainsNarrativelyConsiste`
Проверка охранного условия G_2 (медленное движение, граничное значение включено)	`SlowBoundaryIsInclusive`
Проверка охранного условия G_3 (неуклонное сближение выполняется)	`MovementSteadyApproachIsDetected`
Проверка охранного условия G_3 (нарушение монотонности отклоняется)	`NonMonotonicApproachIsRejected`
$S_1 \to S_2$ при $G_1 \wedge G_2 \wedge G_3$	`BaseScenarioResolvesToCameraEffect`, `InferenceCauseIsCameraEffectWhenEventOccurs`, `CognitiveStatesFollowExpectedOrder`, `ImmaterialHeadScenarioRemainsNarrativelyConsiste`
Внутренние когнитивные этапы в S_2: `SURPRISED -> REASONING -> CONCLUDED`	`CognitiveStatesFollowExpectedOrder`
$S_1 \to S_3$ при $\neg(G_1 \wedge G_2 \wedge G_3)$ (ветка “не наезд камеры”)	`NoCameraDollyNoIllusoryPerception`
Инвариант предметной области (бесплотная голова не приводит к физической причине)	`ImmaterialHeadScenarioRemainsNarrativelyConsiste`

Переходы вне текущего покрытия

- Переход в S_3 при активном `DOLLY_IN`, но нарушении только условия G_2 (слишком быстрое движение), явно не проверяется.
- Переход в S_3 при `distance_trace` длины меньше 2 (условие G_3 ложно), явно не проверяется.

Реализованные тесты:

```
TEST(DomainModelTest, BaseScenarioResolvesToCameraEffect) {
    dmn::Arthur arthur{dmn::Head{dmn::Hair{}, dmn::Materiality::IMMATERIAL}};
    arthur.beginMovingToConsole(dmn::MovementToConsole{0.4, {10.0, 8.0, 6.0, 4.0}});
    arthur.processFilmingScene(dmn::CameraShot{dmn::CameraShotType::DOLLY_IN, true});

    const dmn::PerceptionEvent &event = arthur.getLastPerception();
    const dmn::Inference &inference = arthur.getLastInference();

    EXPECT_TRUE(event.hasOccurred());
    EXPECT_TRUE(event.isIllusory());
    EXPECT_EQ(event.getType(), dmn::PerceptionType::HAIR_STIRRING);
    EXPECT_EQ(inference.getCause(), dmn::CauseHypothesis::CAMERA_EFFECT);
}

TEST(DomainModelTest, NoCameraDollyNoIllusoryPerception) {
    dmn::Arthur arthur{dmn::Head{dmn::Hair{}, dmn::Materiality::IMMATERIAL}};
    arthur.beginMovingToConsole(dmn::MovementToConsole{0.4, {10.0, 8.0, 6.0, 4.0}});
    arthur.processFilmingScene(dmn::CameraShot{dmn::CameraShotType::NONE, false});

    const dmn::PerceptionEvent &event = arthur.getLastPerception();
    const dmn::Inference &inference = arthur.getLastInference();

    EXPECT_FALSE(event.hasOccurred());
    EXPECT_FALSE(event.isIllusory());
    EXPECT_EQ(event.getType(), dmn::PerceptionType::NONE);
    EXPECT_EQ(inference.getCause(), dmn::CauseHypothesis::UNDETERMINED);
}

TEST(DomainModelTest, InferenceCauseIsCameraEffectWhenEventOccurs) {
    dmn::Arthur arthur{dmn::Head{dmn::Hair{}, dmn::Materiality::IMMATERIAL}};
    arthur.beginMovingToConsole(dmn::MovementToConsole{0.7, {12.0, 9.0, 7.0, 5.0}});
    arthur.processFilmingScene(dmn::CameraShot{dmn::CameraShotType::DOLLY_IN, true});

    ASSERT_TRUE(arthur.getLastPerception().hasOccurred());
    EXPECT_EQ(arthur.getLastInference().getCause(), dmn::CauseHypothesis::CAMERA_EFFECT);
}

TEST(DomainModelTest, CognitiveStatesFollowExpectedOrder) {
    dmn::Arthur arthur{dmn::Head{dmn::Hair{}, dmn::Materiality::IMMATERIAL}};
    arthur.beginMovingToConsole(dmn::MovementToConsole{0.6, {9.0, 7.0, 5.0, 3.0}});
    arthur.processFilmingScene(dmn::CameraShot{dmn::CameraShotType::DOLLY_IN, true});

    EXPECT_EQ(arthur.getLastInference().getTimeline(),
              (std::vector<dmn::CognitiveState>{dmn::CognitiveState::SURPRISED, dmn::Cogni
                                                dmn::CognitiveState::CONCLUDED}));
}

TEST(DomainModelTest, MovementSteadyApproachIsDetected) {
    const dmn::MovementToConsole movement{0.5, {6.0, 5.0, 4.0, 4.0, 2.0}};

    EXPECT_TRUE(movement.isSteadyApproach());
}

```

```
TEST(DomainModelTest, SlowBoundaryIsInclusive) {
    const dmn::MovementToConsole movement{dmn::MovementToConsole::K_SLOW_SPEED_UPPER_BOUND

    EXPECT_TRUE(movement.isSlow());
}

TEST(DomainModelTest, NonMonotonicApproachIsRejected) {
    const dmn::MovementToConsole movement{0.5, {6.0, 4.0, 5.0, 3.0}};

    EXPECT_FALSE(movement.isSteadyApproach());
}

TEST(DomainModelTest, ImmaterialHeadScenarioRemainsNarrativelyConsistent) {
    dmn::Arthur arthur{dmn::Head{dmn::Hair{}, dmn::Materiality::IMMATERIAL}};
    arthur.beginMovingToConsole(dmn::MovementToConsole{0.4, {8.0, 6.0, 4.0, 2.0}});
    arthur.processFilmingScene(dmn::CameraShot{dmn::CameraShotType::DOLLY_IN, true});

    EXPECT_TRUE(arthur.getLastPerception().hasOccurred());
    EXPECT_EQ(arthur.getLastInference().getCause(), dmn::CauseHypothesis::CAMERA_EFFECT);
    EXPECT_TRUE(arthur.isNarrativeConsistent());
}
```

Выводы

Я ознакомился с основными методами тестирования программного обеспечения и научился работать с библиотекой Google Test